

Pontificia Universidad Católica de Chile

EPG4001 Aprendizaje Supervisado

Material Complementario

Clase5MIAApSup.pdf

Jorge Luis Bazán
jlbazan@uc.cl

Librerías

Las librerías a utilizar para los procedimientos son las siguientes:

```
library(MASS)           # lda(), qda()
library(e1071)          # naiveBayes()
library(naivebayes)     # naive_bayes()
library(ISLR2)          # Bases de datos ISL
library(caret)          # confusionMatrix(), trainControl()
library(ggplot2)        # Visualización
library(tidyverse)      # Manipulación de datos
library(knitr)          # Tablas
```

1. Naive Bayes

Naive Bayes es un clasificador generativo que estima la probabilidad posterior $P(Y = k \mid \mathbf{x})$ mediante el teorema de Bayes, asumiendo independencia condicional de los predictores dado el grupo. A pesar de su supuesto simplificador, suele tener buen desempeño empírico, especialmente cuando p es grande o el tamaño muestral es pequeño.

1.1. Ejemplo manual

Se considera la siguiente muestra de $n = 10$ observaciones con dos predictores continuos x_1 , x_2 y una respuesta binaria y :

```
datos <- data.frame(
  y = factor(c(1, 0, 0, 1, 1, 1, 1, 1, 0, 0)),
  x1 = c(9, 6, 1, 3, 3, 1, 9, 9, 4, 6),
  x2 = c(6, 2, 6, 9, 7, 9, 8, 2, 6, 4)
)

# Datos de entrenamiento :
datos
```

```
##      y x1 x2
## 1    1  9  6
## 2    0  6  2
## 3    0  1  6
## 4    1  3  9
## 5    1  3  7
## 6    1  1  9
## 7    1  9  8
## 8    1  9  2
## 9    0  4  6
## 10   0  6  4
```

```
kable(t(datos), caption = "Muestra de entrenamiento")
```

Cuadro 1: Muestra de entrenamiento

y	1	0	0	1	1	1	1	1	0	0
x1	9	6	1	3	3	1	9	9	4	6
x2	6	2	6	9	7	9	8	2	6	4

El objetivo es clasificar la nueva observación $(x_1, x_2) = (4, 8)$.

1.1.1. Estadísticas por grupo

Se calculan las proporciones muestrales $\hat{\pi}_k$ y las medias y desviaciones estándar de cada predictor por grupo, que servirán como parámetros de las densidades Gaussianas marginales.

```
resumen <- datos %>%
  group_by(y) %>%
  summarise(
    n      = n(),
    pi_hat = n() / nrow(datos),
    mu_x1  = mean(x1), sd_x1 = sd(x1),
    mu_x2  = mean(x2), sd_x2 = sd(x2)
  )

# Estadísticas muestrales por grupo :
resumen

## # A tibble: 2 x 7
##   y      n pi_hat mu_x1 sd_x1 mu_x2 sd_x2
##   <fct> <int> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 0         4    0.4  4.25  2.36  4.5    1.91
## 2 1         6    0.6  5.67  3.72  6.83   2.64

kable(resumen, digits = 4,
      caption = "Estadísticas muestrales por grupo")
```

Cuadro 2: Estadísticas muestrales por grupo

y	n	pi_hat	mu_x1	sd_x1	mu_x2	sd_x2
0	4	0.4	4.2500	2.3629	4.5000	1.9149
1	6	0.6	5.6667	3.7238	6.8333	2.6394

1.1.2. Cálculo manual con marginales Gaussianas

Bajo Naive Bayes con densidades marginales Gaussianas, la densidad conjunta condicional al grupo j se factoriza como:

$$\hat{f}_j(\mathbf{x}) = \prod_{k=1}^p \phi(x_k; \bar{x}_{jk}, S_{jk}^2)$$

donde $\phi(\cdot; \mu, \sigma^2)$ es la densidad Normal con media μ y varianza σ^2 . La probabilidad posterior se obtiene entonces como:

$$P(G = j \mid \mathbf{x}) = \frac{\hat{\pi}_j \hat{f}_j(\mathbf{x})}{\sum_{\ell} \hat{\pi}_{\ell} \hat{f}_{\ell}(\mathbf{x})}$$

```
# Parámetros por grupo
mu1 <- c(5.6667, 6.8333); sd1 <- c(3.7239, 2.6394)
mu2 <- c(4.2500, 4.5000); sd2 <- c(2.3629, 1.9149)
pi1 <- 6/10; pi2 <- 4/10
xnew <- c(4, 8)

f1 <- prod(dnorm(xnew, mu1, sd1))
f2 <- prod(dnorm(xnew, mu2, sd2))

post1 <- (pi1 * f1) / (pi1 * f1 + pi2 * f2)
post2 <- (pi2 * f2) / (pi1 * f1 + pi2 * f2)

# Densidad conjunta grupo 1, f1(4,8):
round(f1, 6)
```

```
## [1] 0.013286
```

```
# Densidad conjunta grupo 2, f2(4,8):
round(f2, 6)
```

```
## [1] 0.006582
```

```
# P(G=1 | x):
round(post1, 4)
```

```
## [1] 0.7517
```

```
# P(G=2 | x):
round(post2, 4)
```

```
## [1] 0.2483
```

La observación se clasifica en $G = 1$ (grupo $Y = 1$) con probabilidad posterior $\approx 0,75$, lo que indica una preferencia clara hacia dicha clase dada la nueva observación.

1.2. Ajuste con `e1071::naiveBayes`

Se ajusta el modelo Naive Bayes con la función `naiveBayes()` del paquete `e1071` y se predicen las probabilidades posteriores para la nueva observación $(x_1, x_2) = (4, 8)$:

```
mod_nb <- naiveBayes(y ~ x1 + x2, data = datos)
pred_nb <- predict(mod_nb, newdata = data.frame(x1 = 4, x2 = 8),
                  type = "raw")

# Probabilidades posteriores :
round(pred_nb, 4)
```

```
##           0      1
## [1,] 0.2483 0.7517
```

```
kable(round(pred_nb, 4),
      caption = "Probabilidades posteriores — e1071::naiveBayes")
```

Cuadro 3: Probabilidades posteriores — e1071::naiveBayes

0	1
0.2483	0.7517

1.3. Ajuste con `naivebayes::naive_bayes`

El paquete `naivebayes` ofrece una implementación alternativa con mayor flexibilidad de densidades. Se verifica la consistencia con el cálculo manual y con `e1071`:

```
mod_nb2 <- naive_bayes(y ~ x1 + x2, data = datos)
pred_nb2 <- predict(mod_nb2,
                    newdata = data.frame(x1 = 4, x2 = 8),
                    type = "prob")

# Probabilidades posteriores :
round(pred_nb2, 4)
```

```
##           0      1
## [1,] 0.2483 0.7517
```

```
kable(round(pred_nb2, 4),
      caption = "Probabilidades posteriores — naivebayes::naive_bayes")
```

Cuadro 4: Probabilidades posteriores — naivebayes::naive_bayes

0	1
0.2483	0.7517

Ambas implementaciones coinciden con el cálculo manual, confirmando la correcta aplicación del teorema de Bayes bajo el supuesto de independencia condicional.

2. Análisis Discriminante Lineal (LDA)

El LDA es un clasificador generativo que asume que las observaciones de cada clase provienen de una distribución Gaussiana multivariada con la **misma** matriz de covarianza Σ para todos los grupos. Esta restricción produce fronteras de decisión lineales en el espacio de predictores. La regla de clasificación asigna la observación al grupo k que maximiza la función discriminante lineal:

$$\delta_k(\mathbf{x}) = \mathbf{x}^\top \Sigma^{-1} \hat{\boldsymbol{\mu}}_k - \frac{1}{2} \hat{\boldsymbol{\mu}}_k^\top \Sigma^{-1} \hat{\boldsymbol{\mu}}_k + \log(\hat{\pi}_k)$$

2.1. Ejemplo manual

Usando los mismos datos, clasificamos (4,8) mediante LDA, que asume $\Sigma_1 = \Sigma_2 = \Sigma$ (covarianza pooled).

2.1.1. Estimación de parámetros

Se estiman las medias por grupo y la matriz de covarianza pooled, que pondera las covarianzas muestrales de cada clase por sus grados de libertad:

```
G1 <- datos[datos$y == 1, c("x1", "x2")]
G2 <- datos[datos$y == 0, c("x1", "x2")]

mu_hat1 <- colMeans(G1)
mu_hat2 <- colMeans(G2)

S1 <- cov(G1) # Sigma_1
S2 <- cov(G2) # Sigma_2
n1 <- nrow(G1); n2 <- nrow(G2); n <- n1 + n2

# Pooled covariance
S_pool <- ((n1 - 1) * S1 + (n2 - 1) * S2) / (n - 2)
S_inv <- solve(S_pool)

# Media estimada grupo 1:
round(mu_hat1, 4)
```

```
##      x1      x2
## 5.6667 6.8333
```

```
# Media estimada grupo 2:
round(mu_hat2, 4)
```

```
##      x1      x2
## 4.25 4.50
```

```
# Covarianza pooled:
round(S_pool, 4)
```

```
##           x1      x2
## x1 10.7604 -5.2292
## x2 -5.2292  5.7292
```

```
# Inversa de Sigma pooled:
round(S_inv, 4)
```

```
##           x1      x2
## x1 0.1670 0.1524
## x2 0.1524 0.3137
```

2.1.2. Funciones discriminantes lineales

Con los parámetros estimados se evalúan las funciones δ_k en la nueva observación y se calculan las probabilidades posteriores correspondientes:

```
xnew <- c(4, 8)
pi1 <- n1 / n; pi2 <- n2 / n

delta1 <- as.numeric(xnew %*% S_inv %*% mu_hat1 -
                     0.5 * mu_hat1 %*% S_inv %*% mu_hat1 + log(pi1))
delta2 <- as.numeric(xnew %*% S_inv %*% mu_hat2 -
                     0.5 * mu_hat2 %*% S_inv %*% mu_hat2 + log(pi2))

# delta_1(4,8):
round(delta1, 4)
```

```
## [1] 15.5919
```

```
# delta_2(4,8):
round(delta2, 4)
```

```
## [1] 13.5423
```

```
# P(G=1 | x):
round(exp(delta1) / (exp(delta1) + exp(delta2)), 4)
```

```
## [1] 0.8859
```

```
# P(G=2 | x):
round(exp(delta2) / (exp(delta1) + exp(delta2)), 4)
```

```
## [1] 0.1141
```

Como $\delta_1 > \delta_2$, la observación se clasifica en $G = 1$, resultado consistente con el obtenido mediante Naive Bayes.

2.2. Ajuste con MASS::lda

Se ajusta el modelo con la función `lda()` del paquete MASS y se predice la clase y las probabilidades posteriores para la nueva observación:

```
mod_lda <- lda(y ~ x1 + x2, data = datos)
pred_lda <- predict(mod_lda, newdata = data.frame(x1 = 4, x2 = 8))

# Clase predicha:
as.character(pred_lda$class)
```

```
## [1] "1"
```

```
# Probabilidades posteriores :
round(pred_lda$posterior, 4)
```

```
##          0          1
## 1 0.1141 0.8859
```

```
kable(round(pred_lda$posterior, 4),
      caption = "Probabilidades posteriores - MASS::lda")
```

Cuadro 5: Probabilidades posteriores — MASS::lda

	0	1
	0.1141	0.8859

2.3. LDA con datos reales: Default (ISLR2)

Aplicamos LDA al conjunto Default para predecir si un cliente incurrirá en impago (`default`) en función de `balance` e `income`. Se divide la muestra en 70 % entrenamiento y 30 % prueba:

```
data(Default, package = "ISLR2")

set.seed(123)
n_def <- nrow(Default)
idx   <- sample(1:n_def, floor(0.7 * n_def))
train <- Default[idx, ]; test <- Default[-idx, ]

mod_lda_def <- lda(default ~ balance + income, data = train)
pred_lda_def <- predict(mod_lda_def, newdata = test)

cm_lda <- confusionMatrix(pred_lda_def$class, test$default,
                          positive = "Yes")

# Accuracy:
round(cm_lda$overall["Accuracy"], 4)
```

```
## Accuracy
##    0.9713
```

```
# Sensibilidad:
round(cm_lda$byClass["Sensitivity"], 4)
```

```
## Sensitivity
##          0.22
```

```
# Especificidad:
round(cm_lda$byClass["Specificity"], 4)
```

```
## Specificity
##          0.9972
```



```
# Matriz de confusión :
cm_lda$table
```

```
##           Reference
## Prediction  No  Yes
##           No 2892  78
##           Yes   8  22
```

```
kable(cm_lda$table,
      caption = "Matriz de Confusión - LDA sobre Default (test 30\\%)")
```

Cuadro 6: Matriz de Confusión — LDA sobre Default (test 30 %)

	No	Yes
No	2892	78
Yes	8	22

El modelo LDA alcanza una accuracy elevada ($\approx 97\%$) gracias a la gran proporción de no-impagos en los datos. Sin embargo, la sensibilidad es baja, lo que refleja la dificultad del modelo para detectar correctamente los casos de impago (clase minoritaria). La especificidad, en cambio, es muy alta.

3. Análisis Discriminante Cuadrático (QDA)

A diferencia de LDA, QDA permite que cada clase tenga su propia matriz de covarianza $\hat{\Sigma}_k$, lo que produce fronteras de decisión cuadráticas. Esto le otorga mayor flexibilidad pero a costa de estimar más parámetros (un total de $K \cdot p(p+1)/2$ en lugar de $p(p+1)/2$).

3.1. Ejemplo manual

QDA usa matrices de covarianza independientes por grupo ($\hat{\Sigma}_k$) y la función discriminante cuadrática:

$$\delta_k(\mathbf{x}) = -\frac{1}{2}(\mathbf{x} - \hat{\boldsymbol{\mu}}_k)^\top \hat{\Sigma}_k^{-1}(\mathbf{x} - \hat{\boldsymbol{\mu}}_k) - \frac{1}{2} \log |\hat{\Sigma}_k| + \log(\hat{\pi}_k)$$

A diferencia de LDA, el término cuadrático en $\mathbf{x}$ no se cancela al comparar grupos, dando lugar a fronteras no lineales.

```
S1_inv <- solve(S1)
S2_inv <- solve(S2)

xc1 <- xnew - mu_hat1; xc2 <- xnew - mu_hat2

delta1_q <- -0.5 * as.numeric(xc1 %*% S1_inv %*% xc1) -
             0.5 * log(det(S1)) + log(pi1)
delta2_q <- -0.5 * as.numeric(xc2 %*% S2_inv %*% xc2) -
             0.5 * log(det(S2)) + log(pi2)

# delta_1 ~ Q(4,8):
round(delta1_q, 4)
```

```
## [1] -2.6561
```

```
#  $\delta_2^Q(4,8)$ :  
round(delta2_q, 4)
```

```
## [1] -5.7704
```

```
# Clasificación:  
ifelse(delta1_q > delta2_q, "G=1", "G=2")
```

```
## [1] "G=1"
```

3.2. Ajuste con MASS::qda

Se ajusta el modelo con la función `qda()` del paquete `MASS` y se verifica que el resultado coincide con el cálculo manual:

```
mod_qda <- qda(y ~ x1 + x2, data = datos)  
pred_qda <- predict(mod_qda, newdata = data.frame(x1 = 4, x2 = 8))
```

```
# Clase predicha:  
as.character(pred_qda$class)
```

```
## [1] "1"
```

```
# Probabilidades posteriores :  
round(pred_qda$posterior, 4)
```

```
##           0           1  
## 1 0.0425 0.9575
```

```
kable(round(pred_qda$posterior, 4),  
      caption = "Probabilidades posteriores - MASS::qda")
```

Cuadro 7: Probabilidades posteriores — MASS::qda

	0	1
	0.0425	0.9575

3.3. Comparación LDA vs QDA: Ejemplo ISL

Reproducimos el experimento de James et al. (2021, §4.4) con datos simulados bivariados en dos escenarios.

Escenario 1: Matrices de covarianza iguales (LDA óptimo)

Primero simulamos los datos

```

set.seed(42)
n_sim <- 150
Sigma_eq <- matrix(c(1, 0.8, 0.8, 1), 2, 2)
mu_A <- c(0, 0); mu_B <- c(2.5, 2.5)

Xa <- MASS::mvrnorm(n_sim, mu_A, Sigma_eq)
Xb <- MASS::mvrnorm(n_sim, mu_B, Sigma_eq)
head(Xa)

```

```

##           [,1]      [,2]
## [1,]  1.31347537  1.2877354
## [2,] -0.04507817 -1.0263613
## [3,] -0.02459756  0.7135853
## [4,]  0.68692055  0.5138518
## [5,]  0.53146829  0.2355769
## [6,]  0.29089121 -0.4922483

```

```
head(Xb)
```

```

##           [,1]      [,2]
## [1,]  2.907921  2.083312
## [2,]  3.300575  3.141883
## [3,]  2.482913  2.591067
## [4,]  3.324938  3.069763
## [5,]  2.327948  2.394140
## [6,]  2.545922  2.344244

```

```

dat1 <- data.frame(
  x1 = c(Xa[,1], Xb[,1]), x2 = c(Xa[,2], Xb[,2]),
  clase = factor(rep(c("A", "B"), each = n_sim))
)
head(dat1)

```

```

##           x1      x2 clase
## 1  1.31347537  1.2877354    A
## 2 -0.04507817 -1.0263613    A
## 3 -0.02459756  0.7135853    A
## 4  0.68692055  0.5138518    A
## 5  0.53146829  0.2355769    A
## 6  0.29089121 -0.4922483    A

```

```
table(dat1$clase)
```

```

##
##  A  B
## 150 150

```

Aplicamos los dos métodos para el conjunto de datos

```
mod_lda1 <- lda(clase ~ x1 + x2, data = dat1)
mod_qda1 <- qda(clase ~ x1 + x2, data = dat1)
```

Haremos una predicción para 200 valores

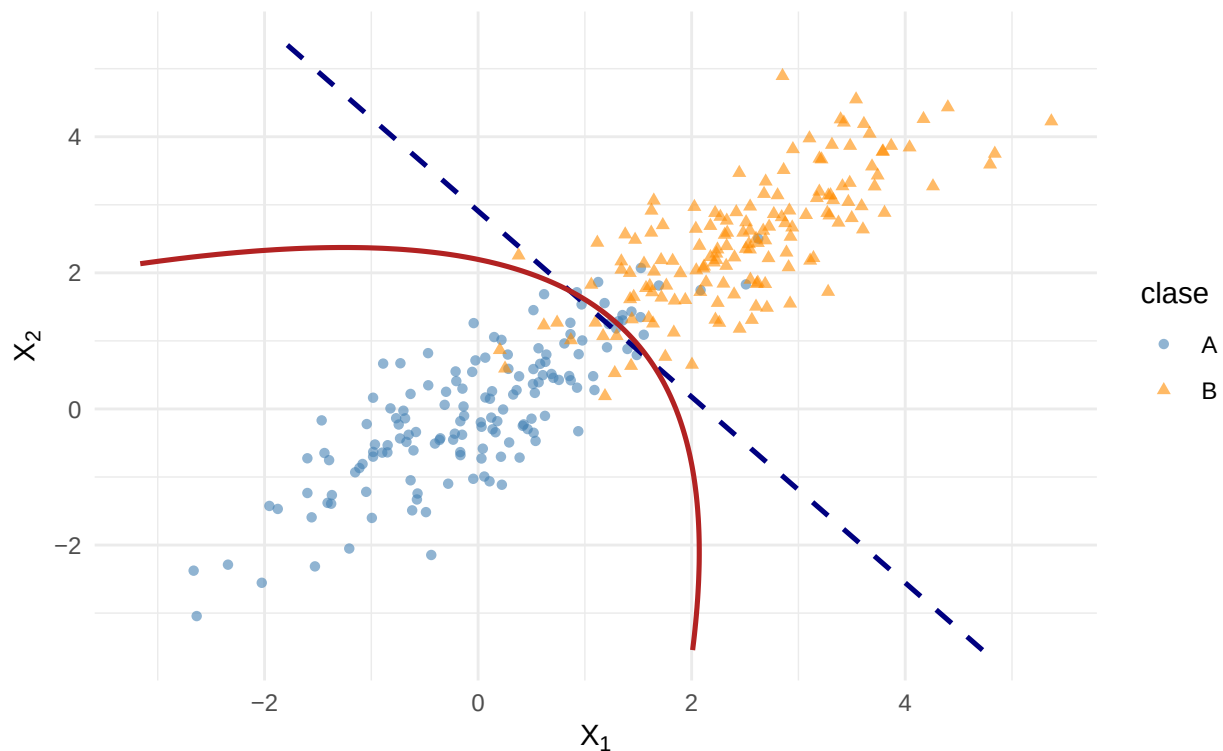
```
# Rejilla de predicción
grid1 <- expand.grid(
  x1 = seq(min(dat1$x1)-0.5, max(dat1$x1)+0.5, length.out = 200),
  x2 = seq(min(dat1$x2)-0.5, max(dat1$x2)+0.5, length.out = 200)
)
grid1$lda_prob <- predict(mod_lda1, grid1)$posterior[, "B"]
grid1$qda_prob <- predict(mod_qda1, grid1)$posterior[, "B"]
```

Ahora representaremos gráficamente los resultados

```
ggplot(dat1, aes(x1, x2, color = clase, shape = clase)) +
  geom_point(alpha = 0.6, size = 1.5) +
  stat_contour(data = grid1, aes(x1, x2, z = lda_prob),
    breaks = 0.5, color = "navy", linetype = "dashed",
    linewidth = 0.9, inherit.aes = FALSE) +
  stat_contour(data = grid1, aes(x1, x2, z = qda_prob),
    breaks = 0.5, color = "firebrick", linetype = "solid",
    linewidth = 0.9, inherit.aes = FALSE) +
  scale_color_manual(values = c("steelblue", "darkorange")) +
  labs(title = "Escenario 1: covarianzas iguales",
    subtitle = "Línea discontinua = LDA | Línea continua = QDA",
    x = expression(X[1]), y = expression(X[2])) +
  theme_minimal(base_size = 11)
```

Escenario 1: covarianzas iguales

Línea discontinua = LDA | Línea continua = QDA



Cuando las covarianzas son iguales las dos fronteras son casi idénticas y la frontera lineal (LDA) es óptima. QDA introduce curvatura innecesaria al estimar parámetros adicionales sin ganancia en sesgo.

Escenario 2: Matrices de covarianza distintas (QDA óptimo)

Primero simulamos datos

```
set.seed(42)
Sigma_A <- matrix(c(1, 0.9, 0.9, 1), 2, 2)
Sigma_B <- matrix(c(3, -1.5, -1.5, 3), 2, 2)

Xa2 <- MASS::mvrnorm(n_sim, mu_A, Sigma_A)
Xb2 <- MASS::mvrnorm(n_sim, mu_B, Sigma_B)
dat2 <- data.frame(
  x1 = c(Xa2[,1], Xb2[,1]), x2 = c(Xa2[,2], Xb2[,2]),
  clase = factor(rep(c("A", "B"), each = n_sim))
)

head(dat2)
```

```
##           x1           x2 clase
## 1  1.34534546  1.3271445     A
## 2 -0.20346373 -0.8973357     A
## 3  0.09294675  0.6149208     A
## 4  0.67802720  0.5556491     A
## 5  0.49864542  0.2894186     A
## 6  0.17344425 -0.3803190     A
```

```
table(dat2$class)
```

```
##  
##   A   B  
## 150 150
```

Aplicamos los dos métodos

```
mod_lda2 <- lda(clase ~ x1 + x2, data = dat2)  
mod_qda2 <- qda(clase ~ x1 + x2, data = dat2)
```

Haremos una predicción para 200 valores

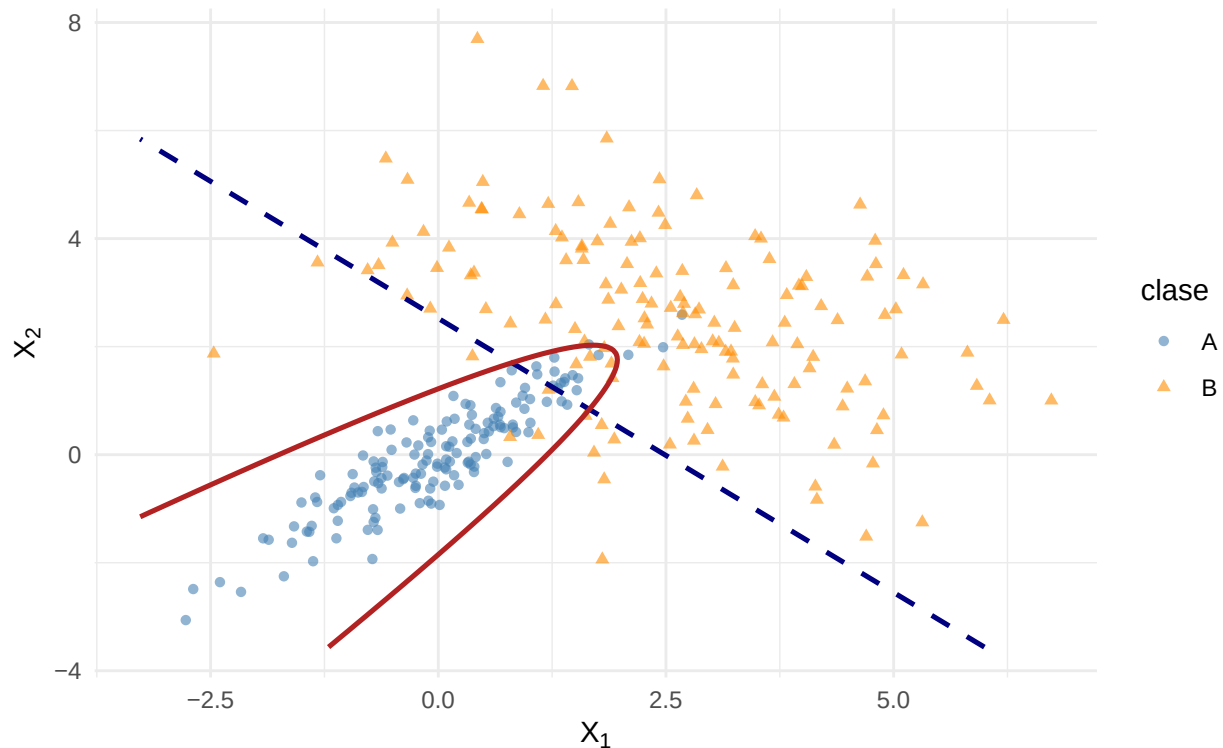
```
grid2 <- expand.grid(  
  x1 = seq(min(dat2$x1)-0.5, max(dat2$x1)+0.5, length.out = 200),  
  x2 = seq(min(dat2$x2)-0.5, max(dat2$x2)+0.5, length.out = 200)  
)  
grid2$lda_prob <- predict(mod_lda2, grid2)$posterior[, "B"]  
grid2$qda_prob <- predict(mod_qda2, grid2)$posterior[, "B"]
```

Ahora representaremos gráficamente los resultados

```
ggplot(dat2, aes(x1, x2, color = clase, shape = clase)) +  
  geom_point(alpha = 0.6, size = 1.5) +  
  stat_contour(data = grid2, aes(x1, x2, z = lda_prob),  
    breaks = 0.5, color = "navy", linetype = "dashed",  
    linewidth = 0.9, inherit.aes = FALSE) +  
  stat_contour(data = grid2, aes(x1, x2, z = qda_prob),  
    breaks = 0.5, color = "firebrick", linetype = "solid",  
    linewidth = 0.9, inherit.aes = FALSE) +  
  scale_color_manual(values = c("steelblue", "darkorange")) +  
  labs(title = "Escenario 2: covarianzas distintas",  
    subtitle = "Línea discontinua = LDA | Línea continua = QDA",  
    x = expression(X[1]), y = expression(X[2])) +  
  theme_minimal(base_size = 11)
```

Escenario 2: covarianzas distintas

Línea discontinua = LDA | Línea continua = QDA



Con covarianzas distintas la frontera QDA (curva) sigue mejor la frontera de Bayes real, mientras que LDA introduce sesgo al asumir covarianzas iguales. La diferencia es especialmente visible en las regiones donde la dispersión del grupo B es mucho mayor.

3.4. QDA con datos reales: Default

```
mod_qda_def <- qda(default ~ balance + income, data = train)
pred_qda_def <- predict(mod_qda_def, newdata = test)
```

```
cm_qda <- confusionMatrix(pred_qda_def$class, test$default,
                           positive = "Yes")
```

```
# Accuracy:
round(cm_qda$overall["Accuracy"], 4)
```

```
## Accuracy
##      0.972
```

```
# Sensibilidad:
round(cm_qda$byClass["Sensitivity"], 4)
```

```
## Sensitivity
##           0.25
```

```
# Especificidad:
round(cm_qda$byClass["Specificity"], 4)
```

```
## Specificity
##      0.9969
```

```
res <- data.frame(
  Método      = c("LDA", "QDA"),
  Accuracy     = round(c(cm_lda$overall["Accuracy"],
                        cm_qda$overall["Accuracy"]), 4),
  Sensibilidad = round(c(cm_lda$byClass["Sensitivity"],
                        cm_qda$byClass["Sensitivity"]), 4),
  Especificidad = round(c(cm_lda$byClass["Specificity"],
                        cm_qda$byClass["Specificity"]), 4)
)
```

```
# Tabla comparativa LDA vs QDA:
res
```

```
##  Método Accuracy Sensibilidad Especificidad
## 1   LDA   0.9713         0.22         0.9972
## 2   QDA   0.9720         0.25         0.9969
```

```
kable(res, caption = "Comparación LDA vs QDA - Default (test 30\\%)",
      booktabs = TRUE, row.names = FALSE)
```

Cuadro 8: Comparación LDA vs QDA — Default (test 30 %)

Método	Accuracy	Sensibilidad	Especificidad
LDA	0.9713	0.22	0.9972
QDA	0.9720	0.25	0.9969

Ambos métodos obtienen accuracy similares. Sin embargo, QDA tiende a mejorar la sensibilidad al capturar mejor la forma de cada grupo, mientras que LDA puede ser preferible cuando el tamaño muestral es pequeño y la estimación de múltiples matrices de covarianza introduce varianza adicional.

4. Ejercicio Propuesto

Ejercicio Considere la base de datos **Smarket** de ISLR2, que contiene 1250 observaciones diarias del mercado bursátil estadounidense entre 2001 y 2005. La variable respuesta es **Direction** (Up / Down) en función de los retornos porcentuales rezagados **Lag1**, **Lag2**, **Lag3**, **Lag4**, **Lag5** y el volumen **Volume**.

- a) Ajuste un modelo **Naive Bayes** (con marginales Gaussianas) utilizando como predictores **Lag1** y **Lag2**. Reporte la matriz de confusión y calcule Accuracy, Sensibilidad, Especificidad y AUC para los datos de entrenamiento (años 2001–2004) y prueba (año 2005).
- b) Repita el ajuste con **LDA** y **QDA** usando los mismos predictores y conjunto de entrenamiento/prueba. Compare las tres métricas de desempeño e indique cuál método preferiría y por qué.
- c) Grafique las fronteras de decisión de LDA y QDA en el espacio (**Lag1**, **Lag2**) e interprete visualmente la diferencia entre ambos métodos.
- d) Usando validación cruzada 10-fold sobre el conjunto de entrenamiento, estime el error de predicción (Accuracy) de los tres métodos. ¿Cambia su conclusión respecto al literal (b)?

Referencias

- [James et al.(2021)] Gareth James, Daniela Witten, Trevor Hastie and Robert Tibshirani (2021) *An Introduction to Statistical Learning with Applications in R*, 2nd edition. Springer Series in Statistics.
- [Hastie et al.(2008)] Trevor Hastie, Robert Tibshirani and Jerome Friedman (2008) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics.